Práctica de Laboratorio 4: RL en Gazebo

DS5345 Aprendizaje por Refuerzo · 2026-II · TurtleBot3 burger · ROS 2 Humble · Gazebo 11 · PyTorch 2.5

> **Código fuente:** https://github.com/artrivas/lab4-rl-gazebo
> **Videos (evidencia Fase 1 y Fase 2):** https://drive.google.com/drive/folders/1dMslm1XBuKaOEY9_Ot0jMoJNVFFXcuxt?usp=sharing

Misión: navegar hasta una **meta fija (1.75, 1.75)** evadiendo 4 cilindros (r = 0.15 m) que orbitan el origen con radio √2 m y periodo 40 s (≈ 0.22 m/s, la misma velocidad máxima del robot), en el mundo *stage 3* de ROBOTIS (arena 4.7 × 4.7 m). El robot parte cerca del origen, por lo que siempre debe cruzar el anillo por donde pasan los obstáculos móviles.

1. Fase 1 – Configuración y validación del simulador

- Opción A: máquina WSL del curso importada (`wsl --import Curso-Robotica C:\WSL\Curso_Robotica entornoLabRL.tar`): Ubuntu 22.04.5, ROS 2 Humble, Gazebo 11, TensorFlow 2.19. Se instaló PyTorch 2.5.1 (CPU) y gymnasium 0.29.1. La PC no tiene GPU NVIDIA (AMD Radeon 780M integrada), por lo que todo corre en CPU (el cuello de botella es la física de Gazebo, no la red).
- **Prueba de validación:** `ros2 launch turtlebot3_gazebo turtlebot3_dqn_stage4.launch.py`, `ros2 run turtlebot3_dqn dqn_gazebo 4`, `dqn_environment` y `dqn_agent ... model_file:=model1.h5`: el robot se mueve en Gazebo y la terminal imprime los *steps*. Con el modelo preentrenado (800 episodios) se continuó el entrenamiento hasta el episodio 1000 (`model2.h5`, `model3.h5`; pesos exportados a .h5/.npz).
- **Hallazgo:** el plugin `libobstacles.so` que anima los cilindros de stage 3/4 sólo existe en `~/rl_ws/build/turtlebot3_gazebo/` y no está en `GAZEBO_PLUGIN_PATH`: Gazebo lo ignora en silencio y **los obstáculos quedan estáticos**. Se corrigió agregando esa ruta (`scripts/ros_env.sh`) y se verificó que el modelo gira ≈ 9°/s.

2. Formulación del MDP (común a las Partes A–C)

Estado $s \in \mathbb{R}^{28}$ (normalizado a [−1, 1]): 24 rayos LiDAR (los 48 del LDS agrupados de a 2 tomando el **mínimo**, / 3.5 m) + distancia a la meta (de `/odom`, / 5 m) + ángulo a la meta relativo al rumbo (/ π) + acción previa (v_{t-1}, ω_{t-1}) normalizada.

Acción continua a = (v, ω) ∈ `gym.spaces.Box([0, -2.84], [0.22, 2.84])` publicada como `geometry_msgs/Twist` en `/cmd_vel`. El actor entrega $u \in \mathbb{R}^2$, que se recorta a [−1, 1] y se re-escala: $v = (u_1+1)/2 \cdot 0.22$ m/s, $\omega = u_2 \cdot 2.84$ rad/s, por lo que nunca se saturan los motores.

Transición: un paso = un scan nuevo (5 Hz en tiempo *simulado*, Δt = 0.2 s). Gracias a esto Gazebo puede correr acelerado (×5, `real_time_update_rate = 0`) sin cambiar lo que ve el agente; la simulación se pausa mientras se actualizan las redes. Episodio: máx. 250 pasos (50 s).

Recompensa (reward shaping):

> $r_t = 20\cdot(d_{t-1} - d_t) - 0.02\cdot\omega_t^2 - 0.005\cdot(\omega_t - \omega_{t-1})^2 - 0.5\cdot\max(0,\, 0.35 - d^{obs}_t)/0.16 - 0.05$; $+100$ si $d_t < 0.25$ m (meta), -100 si $d^{obs}_t < 0.19$ m (colisión)

- Avance continuo hacia la meta (diferencia de potencial: no se puede "farmear" dando vueltas).

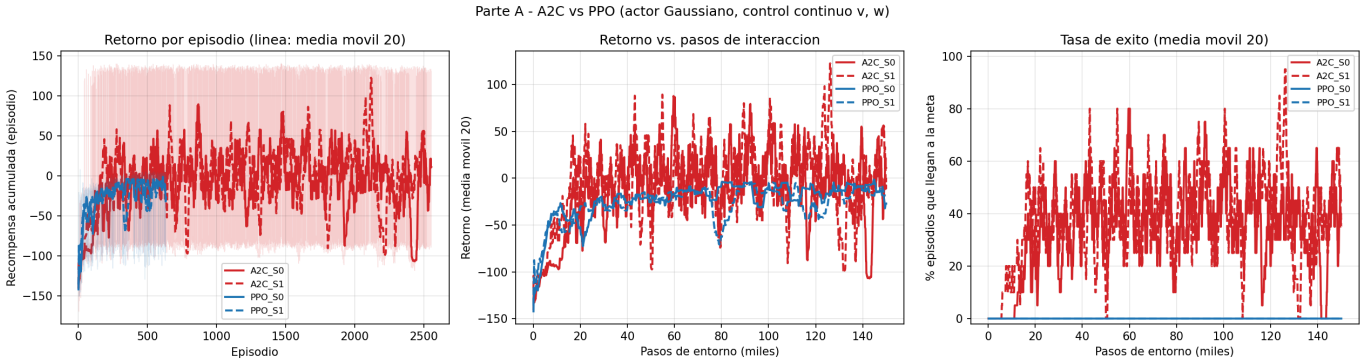
- Penaliza giros bruscos (ω^2) y cambios bruscos de giro; penaliza acercarse a los obstáculos móviles (< 0.35 m) y el tiempo.
- Truncamiento por tiempo con *bootstrap* $\gamma \cdot V(s')$. En entrenamiento las recompensas se escalan ×0.1.

3. Parte A – Control continuo: A2C vs PPO

Actor Gaussiano $\pi_\theta(u|s) = \mathcal{N}(\mu_\theta(s), \text{diag}\,\sigma^2)$: MLP 28→256→256→2 (tanh), $\mu = \tanh(\cdot)$; $\sigma = \exp(\log \sigma)$ con $\log \sigma$ **parámetro aprendible** para v y ω (init −0.5). **Crítico** $V_\varphi(s)$: MLP 28→256→256→1. Adam lr $3 \cdot 10^{-4}$, $\gamma = 0.99$, clip de gradiente 0.5, $c_v = 0.5$, $c_{ent} = 10^{-3}$.

	A2C (estándar)	PPO (clip)
rollout	n = 32 pasos	1024 pasos
ventaja	retornos n-step (GAE λ=1), sin normalizar	GAE λ=0.95, normalizada
pérdida del actor	$-E[\log \pi_\theta(u\|s) \cdot \hat{A}]$	$-E[\min(r_t\hat{A}, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A})]$, $r_t = \pi_\theta/\pi_{\theta_{old}}$, **ε = 0.2**
actualización	1 paso de gradiente por rollout (~4 700 en total)	10 épocas × minibatch 64 (~146 rollouts)

Mismo entorno, red, recompensa y presupuesto (150 000 pasos ≈ 8.3 h simuladas), 2 semillas por algoritmo, 4 Gazebos en paralelo.



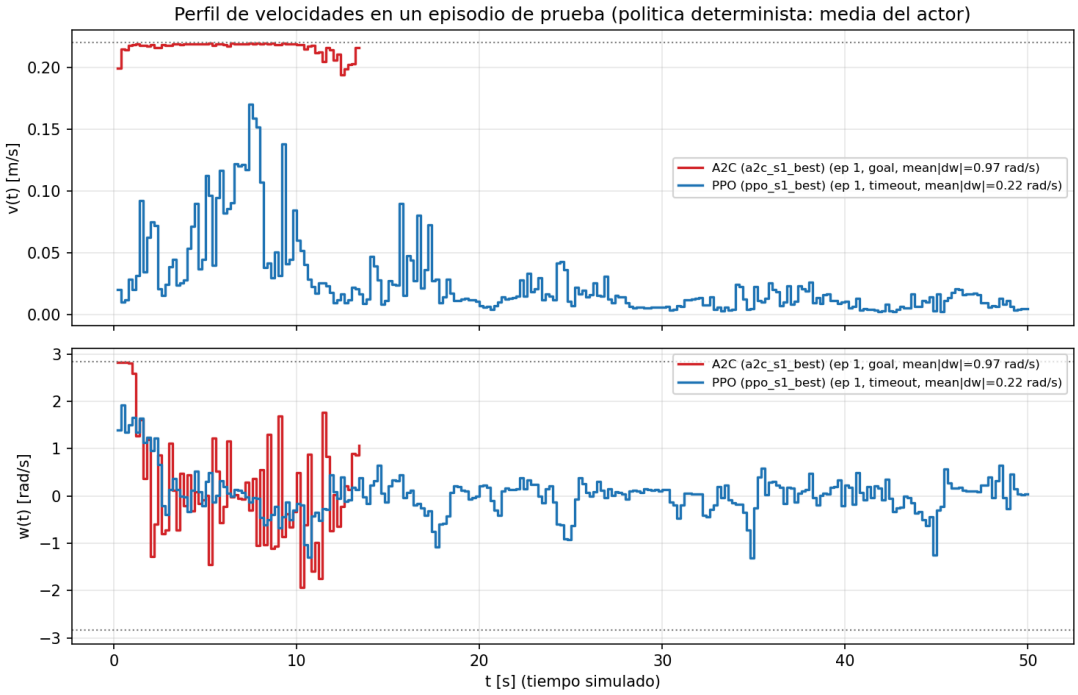
Gráfica (1): recompensa acumulada por episodio (media móvil 20) vs episodios y vs pasos, y tasa de éxito.

corrida	episodios	retorno últimos 50 (media ± desv.)	éxito últimos 50	choque últimos 50	mejor media móvil 20
a2c_s0	2555	-4.7 ± 105.1	38%	62%	87.8
a2c_s1	2522	22.4 ± 109.2	50%	50%	122.7
ppo_s0	634	-15.4 ± 33.3	0%	16%	-1.4
ppo_s1	633	-15.1 ± 31.7	0%	12%	-4.5

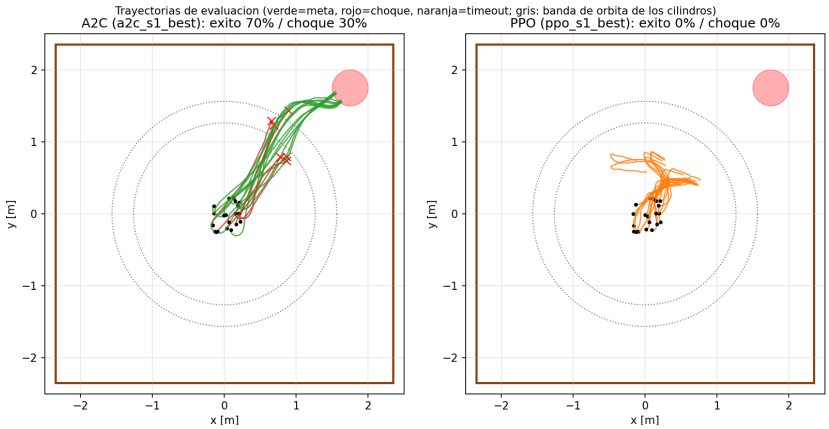
Evaluación (20 episodios, política determinista = media del actor):

actor	éxito	choque	timeout	retorno medio	t medio a meta (s)
a2c_s0_best	70%	30%	0%	71.3	13.2
a2c_s0_last	50%	50%	0%	25.2	12.4
a2c_s1_best → actor_base.pth	70%	30%	0%	73.4	11.8
a2c_s1_last	65%	35%	0%	61.2	13.2

actor	éxito	choque	timeout	retorno medio	t medio a meta (s)
ppo_s0_best	0%	5%	95%	-1.7	–
ppo_s0_last	0%	45%	55%	-41.4	–
ppo_s1_best	0%	0%	100%	0.9	–
ppo_s1_last	0%	0%	100%	-1.3	–



Gráfica (2): perfil temporal v(t), ω(t) en un episodio de prueba.



Trayectorias de evaluación (complementaria).

Análisis

- **A2C** aprende a cruzar el anillo desde ~15k pasos y llega a 40–60 % de éxito estocástico y **70 %** determinista (~12 s a casi $v_{máx}$). Es **inestable**: retorno con desviación ≈ 105–109 y el actor final peor que el mejor — gradiente de política sin restricción del tamaño de paso y ventajas n-step de alta varianza.
- **PPO** es mucho **más estable** (desv. ≈ 32, curvas casi idénticas entre semillas) pero converge a un **óptimo local conservador**: aprende rápido a no chocar (−130 → −15) y luego se queda dentro del anillo, frena (v → 0) y agota el tiempo. El recorte (ε = 0.2), las pocas actualizaciones (~146) y la normalización de ventajas diluyen los pocos éxitos de la exploración inicial, y −100 por choque hace que quedarse quieto sea un máximo local seguro.

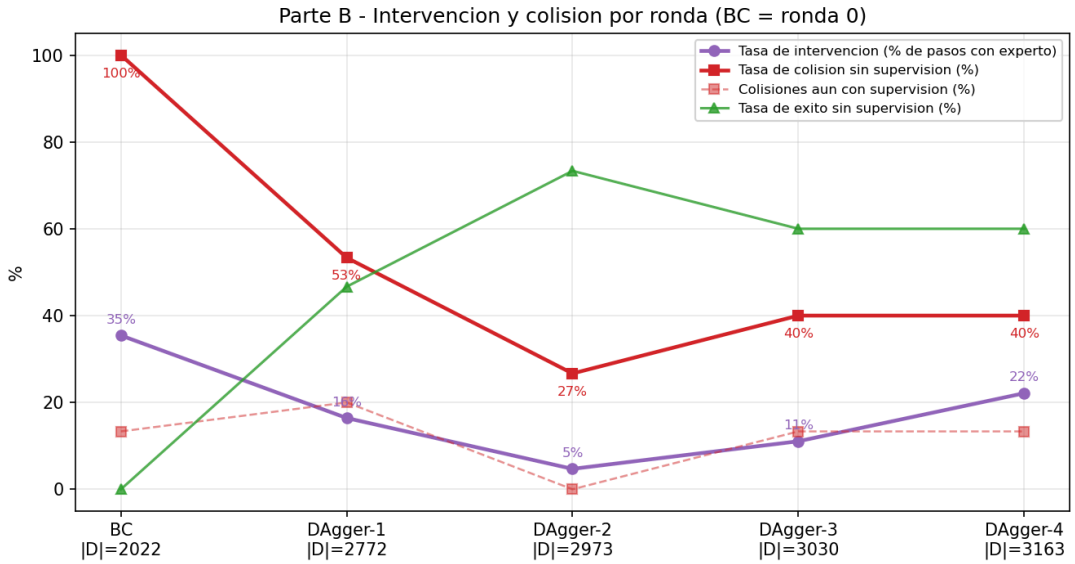
- **Suavidad** (gráfica 2): PPO genera ω(t) mucho más suave (media |Δω| 0.22 vs 0.97 rad/s); A2C corrige el rumbo con giros bruscos porque $-0.02\omega^2$ es pequeño frente al +100 de la meta. Esto motiva las Partes B y C.
- `actor_base.pth` = A2C semilla 1 (mejor éxito y retorno en evaluación).

4. Parte B – Imitation Learning: BC (línea base) vs DAgger

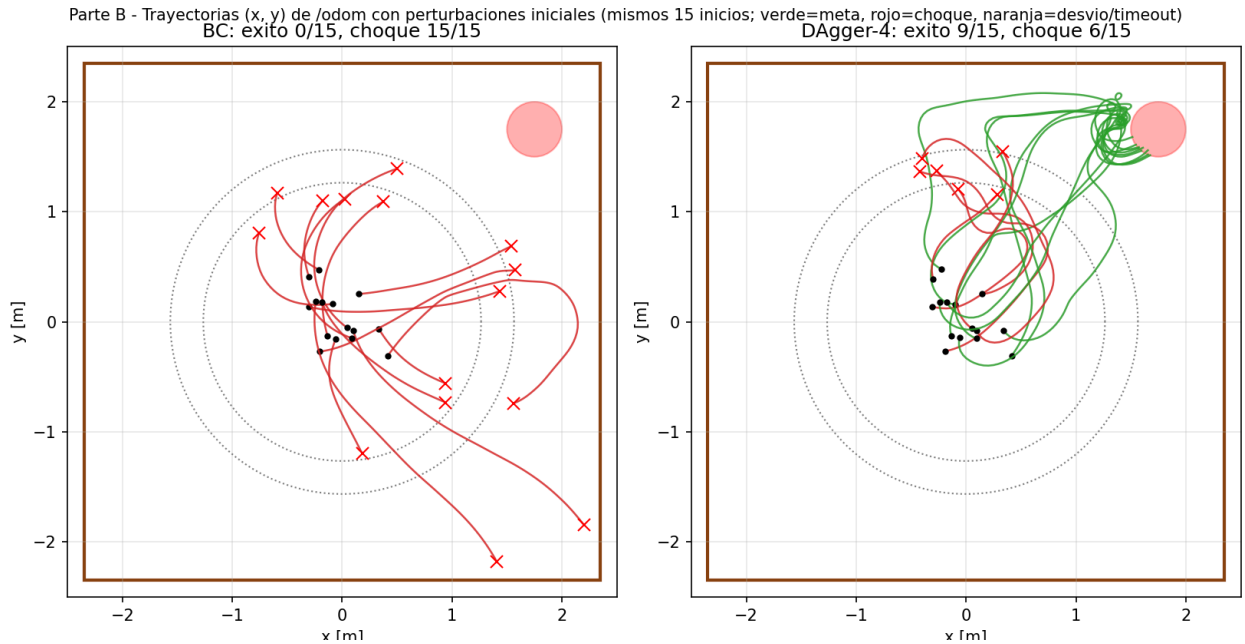
> **Experto / "humano".** Para que las rondas sean reproducibles, el rol del teleoperador lo cumple un **experto oráculo** (`lab4_rl/expert.py`): un planificador con información privilegiada (posición y velocidad de los cilindros, de `/model_states`) que evalúa 102 arcos (v, ω) a 2.4 s, predice la órbita de los obstáculos y elige el que más avanza con margen de seguridad y costo por ω^2 y $\Delta\omega$ (97 % de éxito, $|\Delta\omega| \approx 0.05$ rad/s). El mismo script admite `--expert keyboard`: el agente pilotea y el humano toma el control con el teclado (`i` tomar/soltar, `w/x` v, `a/d` ω), capturando igual los pares ($s_{peligro}$, $a_{experto}$).

- **D_0**: 30 episodios del experto en condiciones "fáciles" (inicio a ±5 cm del origen, mirando a la meta ±0.3 rad) → 2 022 pares (s, a).
- **π_{BC}**: misma arquitectura del actor; regresión MSE de $\mu_\theta(s)$ sobre la acción del experto (200 épocas, MSE validación 0.0054); `bc.pth`.
- **Despliegue con perturbaciones iniciales**: inicio a ±0.5 m y **rumbo aleatorio** (estados nunca vistos en D_0).
- **DAgger interactivo** (4 rondas): el agente pilotea; el experto interviene cuando mantener la acción del agente lleva a un contacto en ~1.2 s (o el robot se queda sin avanzar 5 s). Se agregan sólo los pares de intervención $D \leftarrow D \cup D_i$ y se hace *fine-tuning in situ* (simulación en pausa, 80 épocas) → `dagger_round{i}.pth`, final `dagger_sim.pth`. Cada política se evalúa además sin ayuda en los mismos 15 inicios perturbados.

política	|D|	pares nuevos	tasa de intervención	choques con supervisión	éxito sin supervisión	choque sin supervisión
BC	2022	750	35%	13%	0%	100%
DAgger-1	2772	201	16%	20%	47%	53%
DAgger-2	2973	57	5%	0%	73%	27%
DAgger-3	3030	133	11%	13%	60%	40%
DAgger-4	3163	379	22%	13%	60%	40%



Gráfica (3): tasa de intervención humana y tasa de colisión por ronda de DAgger (ronda 0 = BC).



Gráfica (4): mapa 2D de trayectorias (x, y) de /odom, BC vs DAgger, mismos 15 inicios perturbados.

Análisis

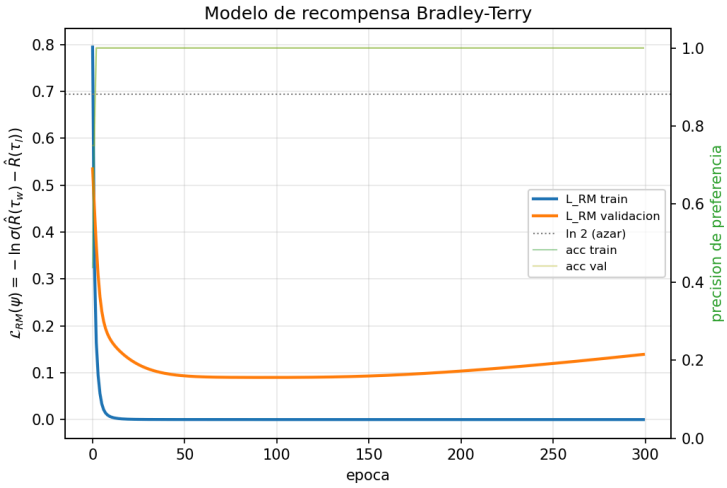
- **Distribution shift:** BC imita muy bien al experto en su distribución (MSE 0.005) pero con rumbos iniciales no vistos sus errores se acumulan: **0/15 éxitos y 15/15 choques**, saliendo en todas direcciones (gráfica 4) — nunca vio cómo recuperarse.
- **DAgger** corrige exactamente esos estados: con 750 pares de la 1ª ronda la colisión baja de 100 % a 53 % y en la 2ª ronda a 27 % con sólo 4.7 % de intervención. Todas las rondas quedan muy por debajo de BC en intervención y colisión.
- La tendencia no es monótona: en las rondas 3–4 la intervención sube (11 % y 22 %) por bloques de ~22 pasos del criterio "sin avanzar" — la política se vuelve más cautelosa (espera a que pasen los cilindros y orbita cerca de la meta antes de entrar). Con 15 episodios por ronda la variabilidad es ±12 pp (la fase de los obstáculos cambia en cada episodio).
- BC gira poco (|ω| medio 0.16 rad/s) y va recto a máxima velocidad hacia donde apunta; DAgger gira más (0.58 rad/s) porque aprendió a reorientarse y esquivar — las correcciones son bruscas, lo que motiva la Parte C.

5. Parte C – Extensión de DAgger con Bradley-Terry y RLHF

Preferencias. Se generaron 20 pares de trayectorias cortas (60 pasos = 12 s, suficiente para llegar al anillo de obstáculos) con el agente DAgger desde el mismo inicio, muestreando con distinto ruido ($\sigma \in \{0.05, 0.25, 0.5, 0.8\}$) para tener evasiones más o menos fluidas. Las etiquetas $\tau_w \succ \tau_l$ las produjo un **etiquetador simulado** con el criterio explícito "más segura y fluida" (sin choque, mayor distancia mínima a obstáculos, menor $\Delta\omega^2$ y ω^2, más avance); `parte_c.py pairs --labeler interactive` permite que un humano elija 1/2 viendo las métricas de cada par.

$$\mathcal{L}_{RM}(\psi) = -\ln \sigma(\hat{R}_\psi(\tau_w) - \hat{R}_\psi(\tau_l)), \quad \hat{R}_\psi(\tau) = \Sigma_t\, r_\psi(s_t, a_t) \qquad R_{RLHF}(s,a) = r_\psi(s,a) - \beta \cdot \ln[\pi_\theta(a|s) / \pi_{DAgger}(a|s)]$$

r_ψ: MLP (28+2)→128→128→1, Adam con weight decay, 16 pares de entrenamiento / 4 de validación, normalizada (media 0, desv. 1). Ajuste final con PPO ($\varepsilon = 0.2$, rollouts de 512, 5 épocas, lr 10^{-4}, σ inicial 0.1 para π_θ y π_{DAgger}) partiendo de π_{DAgger} (referencia congelada), 40 000 pasos para $\beta = 0$, 0.2 y 1.0. `[rm] 16 train / 4 val pares | final train L=0.000 acc=1.00 | val L=0.139 acc=1.00`



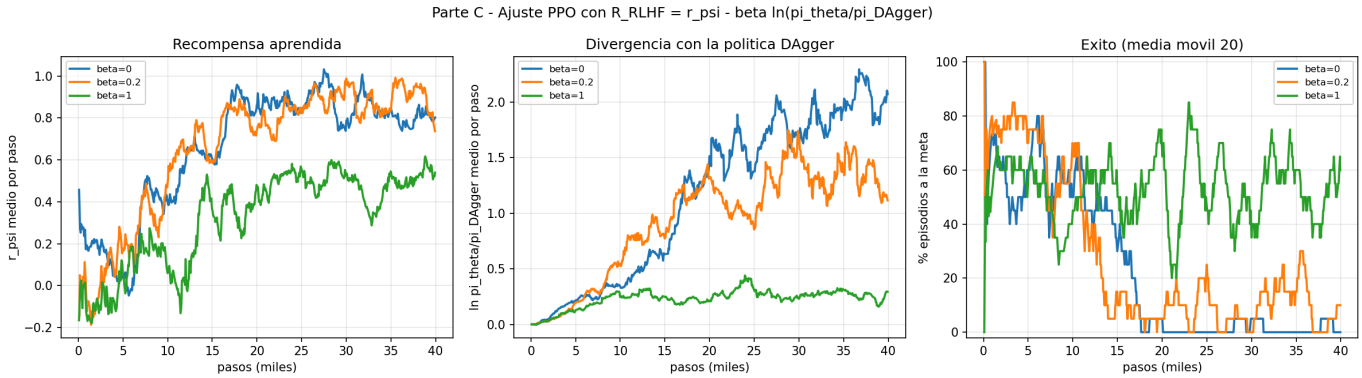
Success rate en 10 episodios (entre parentesis: choques)

politica	ruido LiDAR sigma=0.0 m	ruido LiDAR sigma=0.05 m	ruido LiDAR sigma=0.1 m
BC	0% (100%)	10% (90%)	0% (100%)
DAgger	90% (10%)	60% (40%)	60% (40%)
RLHF_beta0	0% (100%)	0% (90%)	0% (80%)
RLHF_beta0.2	10% (90%)	20% (80%)	20% (70%)
RLHF_beta1	60% (40%)	50% (50%)	80% (20%)

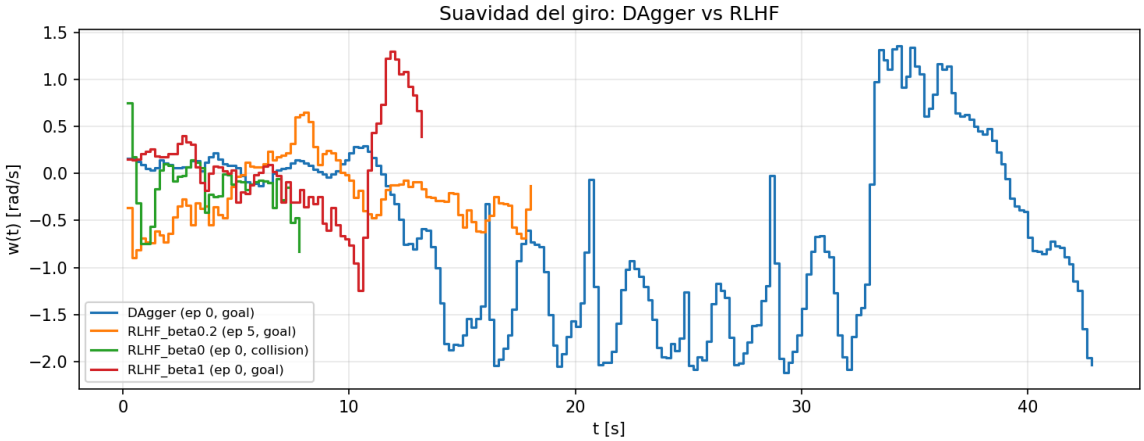
Gráfica (5): curva de pérdida $\mathcal{L}_{RM}$ de Bradley-Terry y tabla de éxito con ruido Gaussiano en el LiDAR.

Prueba de estrés – Success Rate en 10 episodios (mismos inicios perturbados), entre paréntesis la tasa de choque:

política	σ_{LiDAR} = 0.0 m	σ_{LiDAR} = 0.05 m	σ_{LiDAR} = 0.1 m	media $\lvert\Delta\omega\rvert$ sin ruido (rad/s)
BC	0% (100%)	10% (90%)	0% (100%)	0.03
DAgger	90% (10%)	60% (40%)	60% (40%)	0.12
RLHF β=0	0% (100%)	0% (90%)	0% (80%)	0.17
RLHF β=0.2	10% (90%)	20% (80%)	20% (70%)	0.11
RLHF β=1	60% (40%)	50% (50%)	80% (20%)	0.12



Gráfica (6, complementaria): ajuste PPO – recompensa aprendida, divergencia con π_{DAgger} y éxito para cada β.



Gráfica (7, complementaria): ω(t) en un episodio de prueba, DAgger vs RLHF.

Métricas de conducción en la prueba de estrés (sin ruido / σ_{LiDAR} = 0.1 m, promedios por paso sobre los 10 episodios):

política	éxitos	t medio a la meta	ω^2 medio (rad^2/s^2)	|ω| medio	v media (m/s)
BC	0/10 · 0/10	–	0.05 · 0.05	0.17 · 0.19	0.195 · 0.207
DAgger	9/10 · 6/10	24.7 s · 27.5 s	0.65 · 0.75	0.58 · 0.63	0.176 · 0.169
RLHF β = 0	0/10 · 0/10	–	0.32 · 0.40	0.41 · 0.45	**0.219 · 0.219**
RLHF β = 0.2	1/10 · 2/10	18.0 s · 16.4 s	0.15 · 0.12	0.30 · 0.27	0.217 · 0.216
RLHF β = 1	6/10 · **8/10**	**15.1 s · 17.5 s**	**0.34 · 0.37**	0.41 · 0.45	0.213 · 0.216

Análisis

- **Modelo de recompensa:** $\mathscr{L}_{RM}$ de entrenamiento cae a ~0 en ~20 épocas y la de validación alcanza su mínimo (≈ 0.09) cerca de la época 100 y luego sube levemente (sobreajuste con sólo 16 pares); la precisión de preferencia es 100 % en ambos conjuntos (4 pares de validación: evidencia débil). r_ψ aprende sobre todo "avanzar recto y rápido sin girar es mejor", y sólo 5 de los 20 pares contienen un choque.
- **β = 0 → reward hacking.** Sin la penalización KL, PPO maximiza r_ψ sin restricciones: la recompensa aprendida por paso sube de ≈ 0.2 a ≈ 0.85 y la divergencia $\ln \pi_\theta/\pi_{DAgger}$ crece sin control (≈ 2 nats/paso), mientras el éxito real cae de ~70 % a **0 %** (gráfica 6). La política explota el punto ciego del modelo: va siempre a $v_{máx}$ (0.219 m/s) y casi sin evadir, chocando en el 80–100 % de los episodios — "más recompensa aprendida" que no corresponde a la preferencia humana real.
- **β = 0.2** frena parcialmente la deriva (KL ≈ 1.2–1.5) pero no lo suficiente: termina igualmente explotando r_ψ (10–20 % de éxito), aunque es la más suave (ω^2 ≈ 0.13).

- **β = 1 (regularización activa)** mantiene π_θ cerca de la referencia (KL ≈ 0.3 nats/paso) y conserva la capacidad de DAgger de llegar a la meta, pero refinada en la dirección de las preferencias: **la mitad de ω^2** que DAgger (0.34 vs 0.65), llega en **15 s en vez de 25 s** (DAgger orbita alrededor de la meta antes de entrar, gráfica 7) y es la política **más robusta con ruido fuerte** en el LiDAR (80 % de éxito con σ = 0.1 m vs 60 % de DAgger). A cambio pierde éxito nominal sin ruido (60 % vs 90 %). Con 10 episodios por celda la incertidumbre es de ±15 pp, por lo que las diferencias entre DAgger y β = 1 son indicativas, mientras que el colapso de β = 0 y β = 0.2 es claro.
- **Conclusión:** el término $-\beta \ln(\pi_\theta/\pi_{DAgger})$ es lo que hace utilizable un modelo de recompensa entrenado con pocas preferencias: actúa como "ancla" a la política segura de DAgger e impide que el optimizador explote r_ψ. Para mejorar el éxito nominal harían falta más pares (en especial con choques y cruces cercanos), preferencias reales de un humano y/o combinar r_ψ con la recompensa del entorno.

6. Entregables y reproducibilidad

- **Repositorio:** https://github.com/artrivas/lab4-rl-gazebo — entorno ROS 2 + gymnasium (`lab4_rl/env.py`), redes y algoritmos en PyTorch (`agents.py`, `expert.py`, `parte_b.py`, `parte_c.py`), launch y mundo de Gazebo (`launch/`, `worlds/`), scripts de ejecución (`scripts/`) y resultados (`results/`, gráficas en `results/graficas/`).
- **Videos:** https://drive.google.com/drive/folders/1dMslm1XBuKaOEY9_Ot0jMoJNVFFXcuxt?usp=sharing
- **Pesos:** `results/actor_base.pth`, `results/parte_b/bc.pth`, `results/parte_b/dagger_sim.pth`, `results/parte_c/reward_model.pth`, `results/parte_c/rlhf_beta*/actor_rlhf.pth`.
- **Comandos:** `scripts/parte_a_train.sh`, `scripts/parte_a_eval.sh`, `scripts/parte_b.sh`, `scripts/parte_c.sh`; video: `scripts/video_fase1.sh` y `scripts/video_fase2.sh` (Gazebo con GUI en tiempo real).